

Word Embeddings

From Text to Vectors

Firuz Kamalov

MAI 604: Deep Learning

April 29, 2026

Lecture Outline

- 1 Motivation: Representing Data
- 2 Progression of Representations
- 3 Frequency-Based Methods
- 4 Prediction-Based Embeddings
- 5 Contextual Embeddings

The Input Problem

- Machine learning models (especially NNs) only operate on **numerical vectors/tensors**.
- **Tabular data:** Naturally numerical (e.g., age, salary).
- **Images:** Naturally tensors of pixel intensities $[0, 255]$.
- **Text:** Inherently discrete and symbolic.

The Naive (and Invalid) Solution

Why not map words to arbitrary integers?

"cat" = 1, "dog" = 2, "apple" = 3

Why it fails: Models interpret numerical distance and magnitude. This encoding implies that dog is "greater than" cat, and apple is far from cat.

Goal: We need vector representations where **geometry encodes semantics**.

Data Vectorization Across Modalities

1 TABULAR DATA → Vectorized (straightforward)

Example (raw tabular data)

ID	Age	Income	City
1	25	50,000	NY
2	34	64,000	SF
3	28	58,000	LA
...	...	...	...

Vectorization
(direct)

Vectorized representation (numeric matrix)

25	50000	1	0	0
34	64000	0	1	0
28	58000	0	0	1
...	...	...	...	...

Age (numeric)
Income (numeric)
City (one-hot)

2 IMAGE DATA → Vectorized (straightforward)

Example (raw image)



Vectorization
(pixel values)

RGB decomposition

Red channel



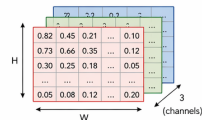
Green channel



Blue channel



Vectorized representation ($H \times W \times 3$ tensor)
A 3D tensor where each location has 3 values (R, G, B)



Shape: $H \times W \times 3$, values in $[0, 1]$ (color intensities per channel)

3 TEXT (SENTENCE) → Vectorized (multiple common ways)

Example (raw text)

The cat sat on the mat

The cat sat on the mat

(a) One-Hot (Bag-of-Words)

Vocabulary = {the, cat, sat, on, mat, dog, ...}

the cat sat on mat dog ...

One-hot vectors (size = $|V|$)

the → [1 0 0 0 0 0 ...]
cat → [0 1 0 0 0 0 ...]
sat → [0 0 1 0 0 0 ...]
on → [0 0 0 1 0 0 ...]
the → [1 0 0 0 0 0 ...]
mat → [0 0 0 0 1 0 ...]

Binary vectors (0/1)

Vectorized representations

(b) TF-IDF

the cat sat on mat dog ...

TF-IDF vectors (size = $|V|$)

the → [0.48 0.32 0.61 0.28 0.47 0.00 ...]
cat → [0.48 0.32 0.61 0.28 0.47 0.00 ...]
sat → [0.48 0.32 0.61 0.28 0.47 0.00 ...]
on → [0.48 0.32 0.61 0.28 0.47 0.00 ...]
the → [0.48 0.32 0.61 0.28 0.47 0.00 ...]
mat → [0.48 0.32 0.61 0.28 0.47 0.00 ...]

Real-valued vectors (TF-IDF weights)

(c) Word Embeddings

(Word2Vec / GloVe)

Embedding dim = 4 (example)

the → [0.21 -0.14 0.35 0.12]
cat → [-0.11 0.23 0.08 -0.31]
sat → [0.05 0.19 -0.27 0.34]
on → [-0.20 0.07 0.18 -0.09]
the → [0.21 -0.14 0.35 0.12]
mat → [-0.15 0.31 -0.10 -0.22]

Each word
mapped to a
dense vector
in $\mathbb{R}^d$

Progression 1: One-Hot Encoding

Definition: Represent each word as a vector in $\mathbb{R}^{|V|}$ with a single 1 and zeroes elsewhere ($|V|$ = vocabulary size).

Toy Example: $V = \{\text{cat}, \text{dog}, \text{apple}\}$

cat $\rightarrow$

1	0	0
---	---	---

dog $\rightarrow$

0	1	0
---	---	---

Properties & Limitations:

- **Extremely high-dimensional** ($|V| \approx 10^5$) and **sparse**.
- **Orthogonal:** The dot product between any two distinct words is exactly 0.
- **Result:** No notion of semantic similarity ($\text{sim}(\text{cat}, \text{dog}) = \text{sim}(\text{cat}, \text{apple})$).

Taxonomy of Text Representations

We can categorize modern text representations into three distinct families:

1. Frequency-Based

Represent documents via word occurrence statistics.

- Bag of Words (BoW)
- TF-IDF
- Co-occurrence Matrices

2. Prediction-Based

Learn dense embeddings via surrogate prediction tasks.

- Word2Vec
- GloVe (*hybrid/count*)
- FastText

3. Contextual

Dynamically adapt the vector based on surrounding text.

- ELMo
- BERT
- GPT

Frequency-Based: Bag of Words (BoW)

Definition: Represent a document (e.g. sentence) as a vector of word counts, ignoring word order.

Numeric Example:

- D_1 : "the cat sat"
- D_2 : "the dog sat"

Vocabulary: $V = \{\text{the, cat, dog, sat}\}$

	the	cat	dog	sat
D_1	1	1	0	1
D_2	1	0	1	1

Limitations: Lack of word-level semantics, sparsity, and common words dominate the vectors artificially.

Bag of Words & TF-IDF Visual Walkthrough

Bag of Words (BoW) Embedding – Example

Each document is represented by a vector that counts (or weights) how many times each word appears in it.

1. Example documents

ID	Document
D1	the cat sat on the mat
D2	the dog sat on the log
D3	the cat chased the dog

2. Build the vocabulary

Collect all unique words and fix an order.

Vocabulary ($V = 7$)

1	the
2	cat
3	sat
4	on
5	mat
6	dog
7	log

3. Convert each document to a BoW vector

Count how many times each vocabulary word appears in the document.

BoW Embedding Matrix (count representation)

Doc / Word	the	cat	sat	on	mat	dog	log
D1	2	1	1	1	1	0	0
D2	2	0	1	1	0	1	1
D3	2	1	0	0	0	1	0

4. (Optional) Use TF-IDF weights instead of counts

TF-IDF reweights common words (like "the") lower.

TF-IDF Embedding Matrix (rounded)

Doc / Word	the	cat	sat	on	mat	dog	log
D1	0.24	0.69	0.69	0.69	0.69	0	0
D2	0.24	0	0.69	0.69	0	0.69	0.69
D3	0.24	0.69	0	0	0	0.69	0

Now each document is represented by a dense vector of size $|V| = 7$.

This vector is the BoW embedding of the document.

How it works

- Create a vocabulary of all unique words.
- For each document, create a vector of length $|V|$.
- The i -th position equals how often the i -th word appears in the document (or its TF-IDF weight).
- This sparse/dense vector is the BoW embedding.

Example: D1 ("the cat sat on the mat")

the	cat	sat	on	mat	dog	log
2	1	1	1	1	0	0

↑ "the" appears twice
↑ "cat" appears once

Frequency-Based: TF-IDF (Mathematical Grounding)

Concept: Balance local frequency (TF) with global rarity (IDF).

$$\text{tfidf}(t, d) = \underbrace{\text{tf}(t, d)}_{\text{count in doc}} \times \underbrace{\log\left(\frac{N}{\text{df}(t)}\right)}_{\text{inverse doc freq}}$$

Applying to the BoW Example ($N = 3$ documents):

- D_1 : "the cat sat on the mat"

Term (t)	tf(D_1)	DF (Global)	tfidf(t, d)	Final (L2-normalized)
the	2	3	$2 \times \log(\frac{3}{3}) = 0$	0.14
cat	1	2	$1 \times \log(\frac{3}{2}) = 0.18$	0.30
mat	1	1	$1 \times \log(\frac{3}{1}) = 0.48$	0.42

Takeaway: Generic structural words ("the") are mathematically downgraded despite having higher raw counts, allowing the actual semantic context ("cat", "mat") to dominate the vector's geometry.

Prediction-Based: Word2Vec (CBOW)

Idea: In the Continuous Bag-of-Words (CBOW) architecture, we learn embeddings by predicting a **center word** based on its surrounding **context words**.

The Mathematical Flow:

- 1 **Context:** Identify surrounding words, e.g., $C = \{w_2, w_3, w_5, w_6\}$ or $\{w_2, w_3\}$.
- 2 **Center:** The target word $\{w_4\}$.
- 3 **Input Embeddings:** Fetch the rows for these words from matrix $\mathbf{U}$.
- 4 **Hidden State:** Average the context vectors into a single vector $\mathbf{h}$.

$$\mathbf{h} = \frac{1}{m} \sum_{w \in C} \mathbf{u}_w$$

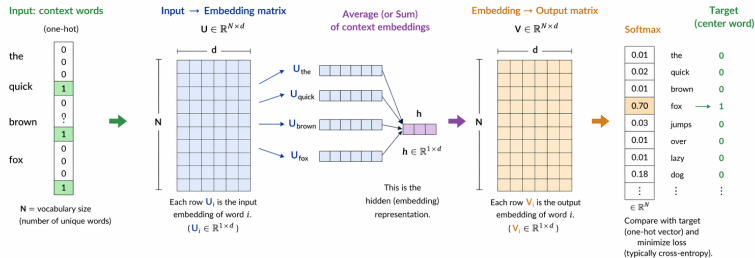
- 5 **Score Candidates:** Calculate the predicted probabilities and use cross-entropy loss .

$$\hat{\mathbf{y}} = \text{Softmax}(\mathbf{h}\mathbf{V}^T)$$

Word2Vec (CBOW): Visual Walkthrough

Word2Vec (CBOW)

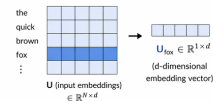
Goal: Given surrounding (context) words, predict the center word.



What are the embeddings?

After training, the input embeddings are the rows of U .

Example: embedding of "fox" = U_{fox}



N = vocabulary size (number of unique words)

d = embedding dimension (number of neurons in hidden layer)

Two sets of embeddings

Word2Vec learns two embeddings for each word:

- Input embedding U_i (row of U) used when word is in the context.
- Output embedding V_i (row of V) used when word is predicted.

After training, we usually keep U (input embeddings) as the word representations.

Mathematical view

For a context set $C = \{w_1, \dots, w_m\}$

- 1) $h = \frac{1}{m} \sum_{i=1}^m U_{w_i} \in \mathbb{R}^{1 \times d}$
- 2) scores = $h V^T \in \mathbb{R}^{1 \times N}$
- 3) $p = \text{softmax}(\text{scores}) \in \mathbb{R}^{1 \times N}$
- 4) Loss = $-\log p(w_{\text{center}})$

Interpretation:

Model tries to make

v_{center}^T h large
and other words' scores small.

(Row v_w^T is the w -th row of V .)

Word2Vec (CBOW): Numeric Mini-Example

Task: Given context "the" and "cat", predict center word "sat".

Vocabulary: $V = \{\text{the, cat, sat, log}\}$ ($N = 4$, embedding dim $d = 2$).

1. Embedding Matrices:

$$U = \begin{bmatrix} 0.4 & 0.2 \\ 0.8 & 0.8 \\ 0.1 & 0.9 \\ -0.5 & -0.1 \end{bmatrix} \begin{matrix} \leftarrow \text{the} \\ \leftarrow \text{cat} \\ \leftarrow \text{sat} \\ \leftarrow \text{log} \end{matrix}, \quad V = \begin{bmatrix} 0.2 & 0.3 \\ 0.7 & 0.6 \\ 1.0 & 1.0 \\ -0.6 & 0.2 \end{bmatrix}$$

2. Input & Hidden State (h):

Look up context rows from U :

- $u_{\text{the}} = [0.4, 0.2]$
- $u_{\text{cat}} = [0.8, 0.8]$

Average them to get h :

$$h = \frac{[0.4, 0.2] + [0.8, 0.8]}{2} = [0.6, 0.5]$$

3. Output Scores over Vocabulary:

Multiply h by V^T to score all words simultaneously:

$$\begin{aligned} s &= hV^T \\ &= [0.6, 0.5] \begin{bmatrix} 0.2 & 0.7 & 1.0 & -0.6 \\ 0.3 & 0.6 & 1.0 & 0.2 \end{bmatrix} \\ &= [0.27, 0.72, \mathbf{1.10}, -0.26] \end{aligned}$$

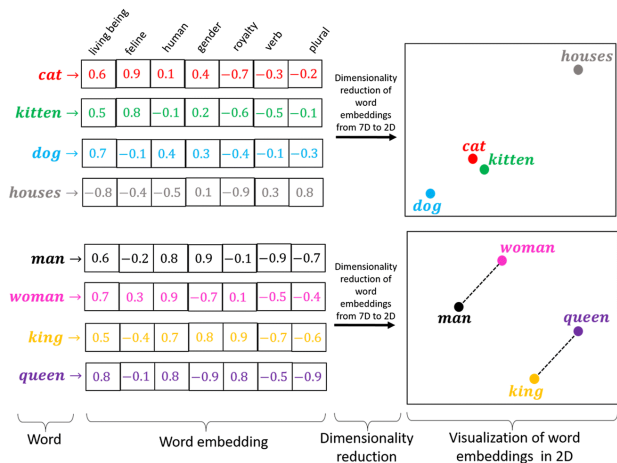
4. Breakdown by Word:

- $\text{soft}(\text{the}) = 0.183$
- $\text{soft}(\text{cat}) = 0.288$
- $\text{soft}(\text{sat}) = \mathbf{0.421}$
- $\text{soft}(\text{log}) = 0.108$

Result: The model correctly assigns the highest pre-softmax score to "sat". Gradient descent will push $v_{\text{sat}}^T h$ even higher.

Geometry of Embeddings

Because Word2Vec relies on inner products, **cosine similarity** naturally captures semantic closeness. Relational meaning is explicitly captured in geometric space!



The Limitation of Word2Vec

The Polysemy Problem

In Word2Vec, GloVe, and FastText, each word type has **exactly one** fixed vector.

Consider the word "**bank**":

- 1 "I sat by the river *bank*."
- 2 "I deposited money in the *bank*."

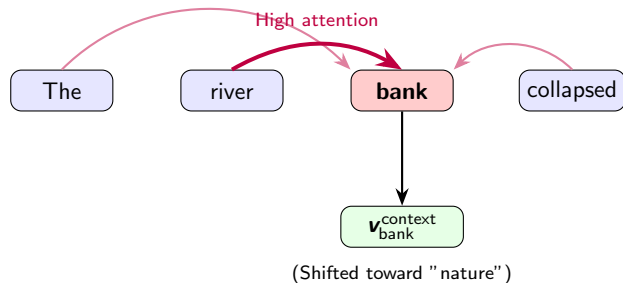
Word2Vec averages these meanings into a single vector. We need a function that maps a word *in its specific context* to a vector:

$$\mathbf{v} = f(\text{word}, \text{context})$$

Enter BERT and GPT (Transformers).

Contextual Mechanism: Self-Attention

Transformers build contextual embeddings layer-by-layer. Tokens "look at" each other to update their representations.



Intuition:

- The word *bank* starts as a raw, static embedding.
- It absorbs meaning from surrounding words via dot-product attention scores.
- The final output vector represents **"bank-as-in-river"** rather than finance.